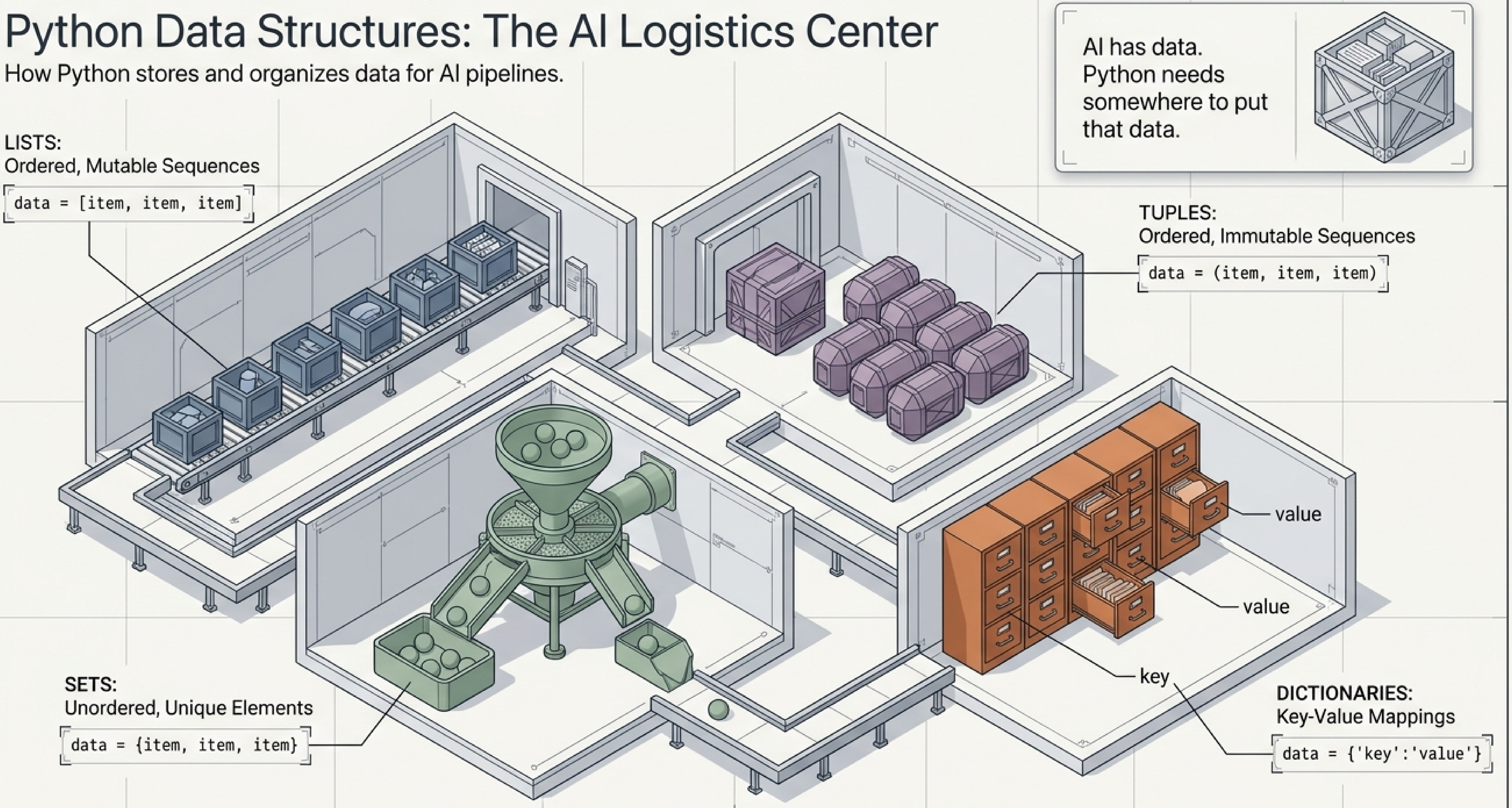
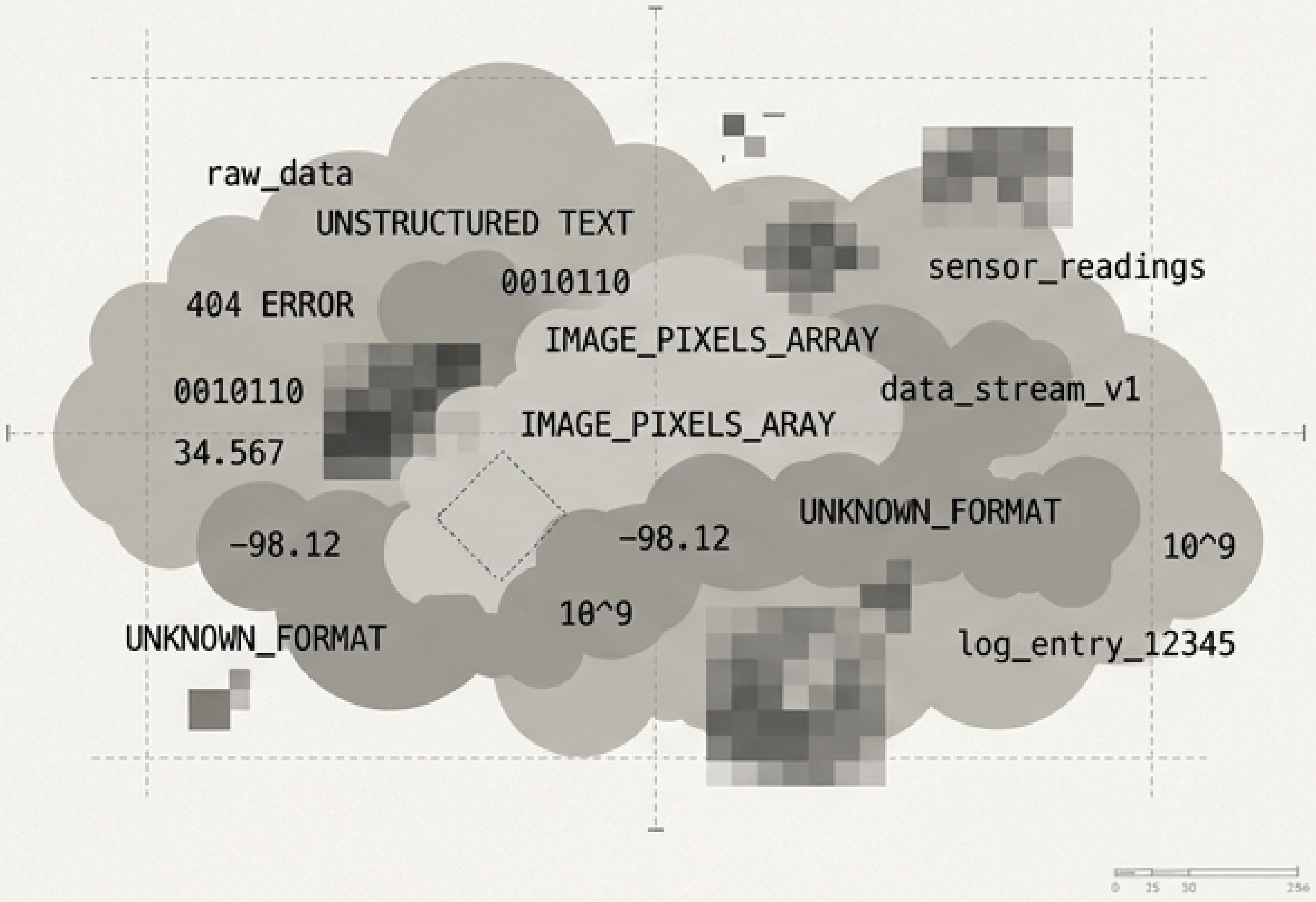
Python Data Structures: The AI Logistics Center

How Python stores and organizes data for AI pipelines.



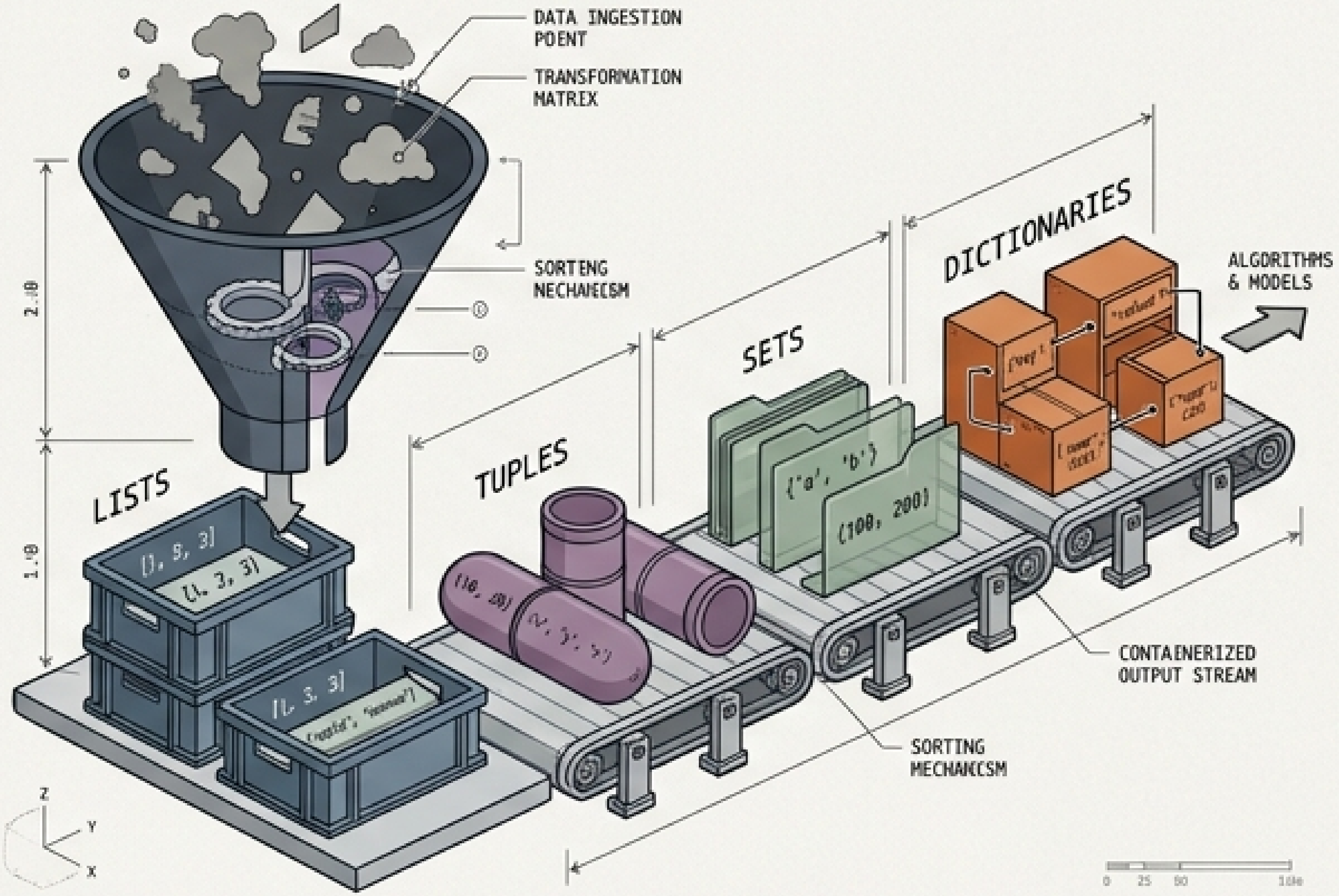
AI systems require rigorous organizational structures to process massive amounts of information

The Problem



Modern AI models consume text, images, and telemetry at scale. Without a logistical framework, Python cannot route this information to the algorithms.

The Solution



Python relies on four major built-in containers to organize this information natively before utilizing advanced libraries.

Lists act as open, ordered crates moving along a predictable conveyor belt

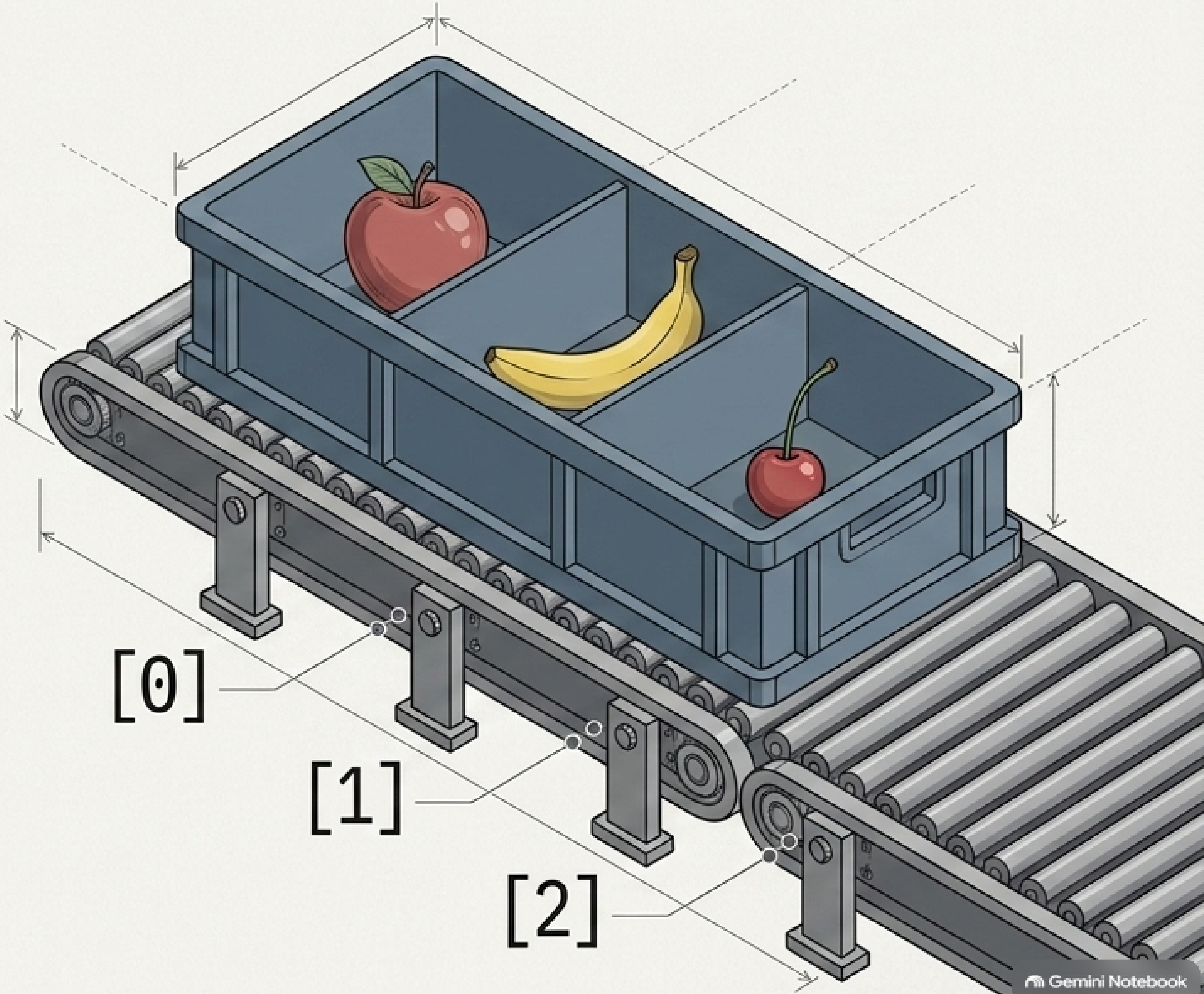
Core Mechanics

- Ordered
- Mutable
- Allows duplicates
- Supports indexing

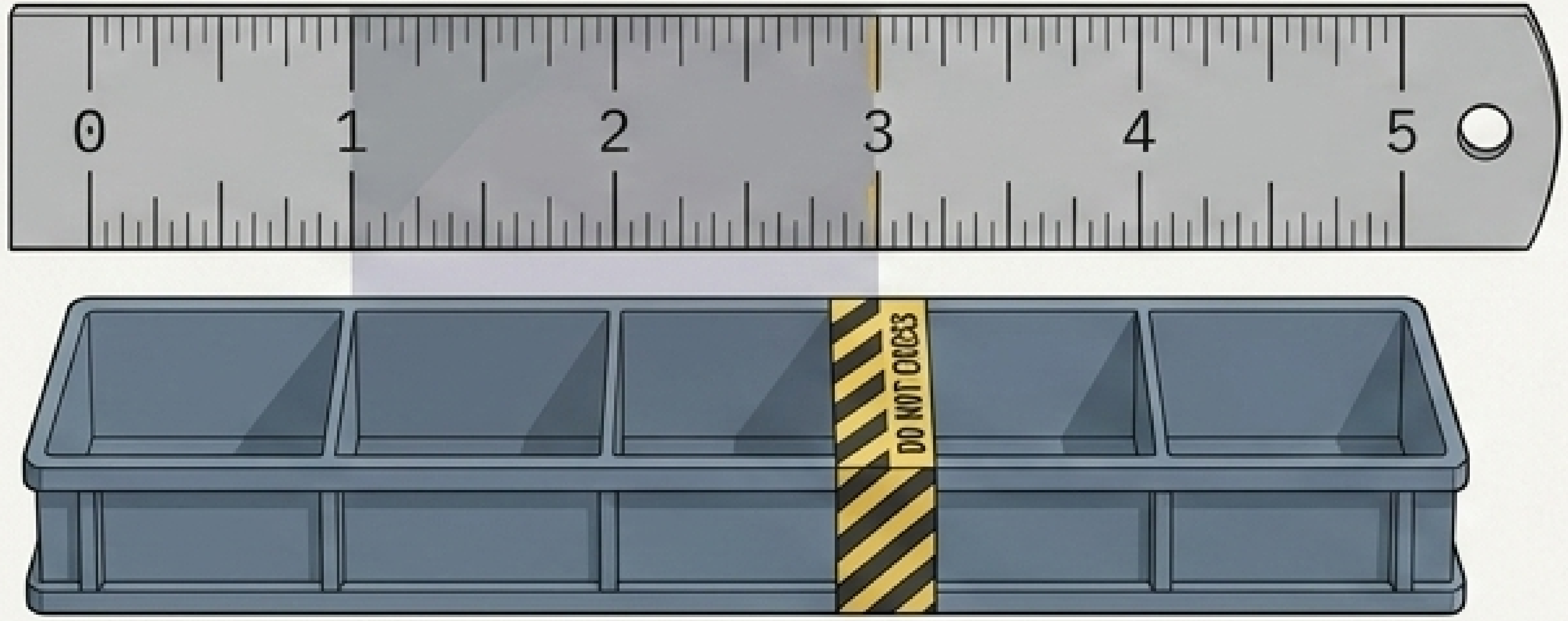
Syntax & Concept

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0]) # Output: apple
```

Because the crate is open and moves linearly, you can always rely on the strict order of its contents.



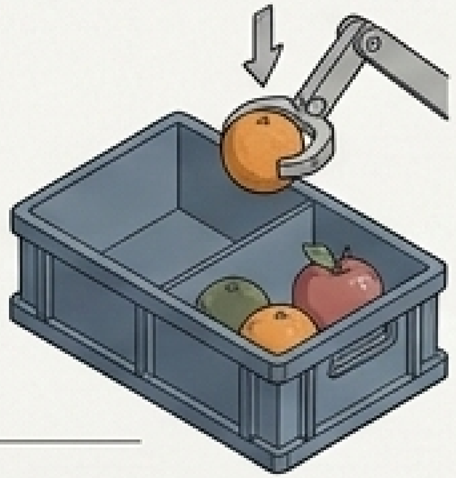
Manipulating list logistics: Modifying contents and slicing exact portions



Slicing Mechanics

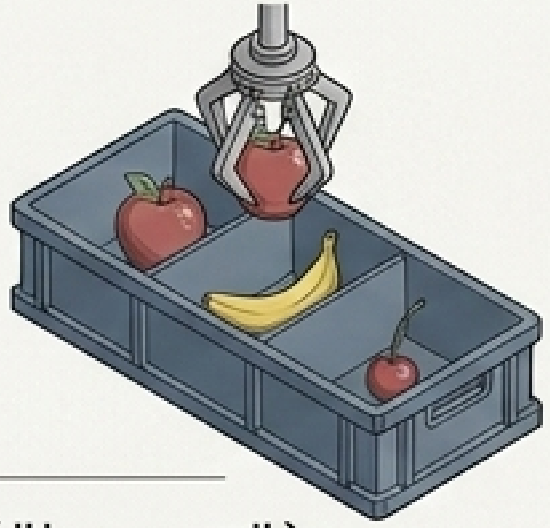
```
Syntax: my_list[start:stop]
Example: nums[1:3] (Grabs index 1 and 2)
```

The stop index is explicitly NOT included.



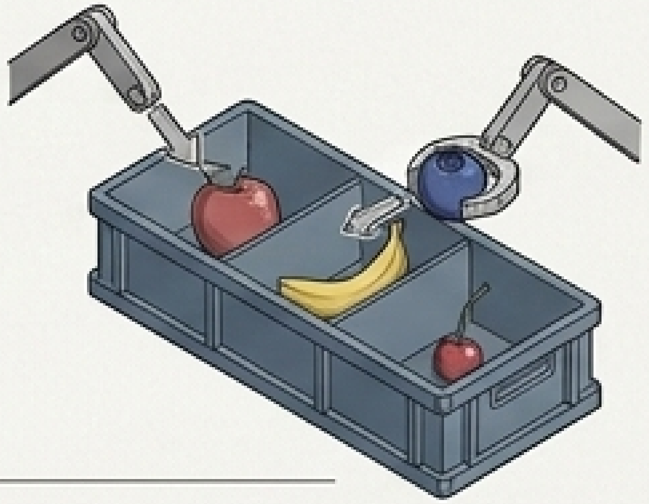
Add

`fruits.append("orange")`
(Drops a new item at the end)



Remove

`fruits.remove("banana")`
(Plucks an item out)

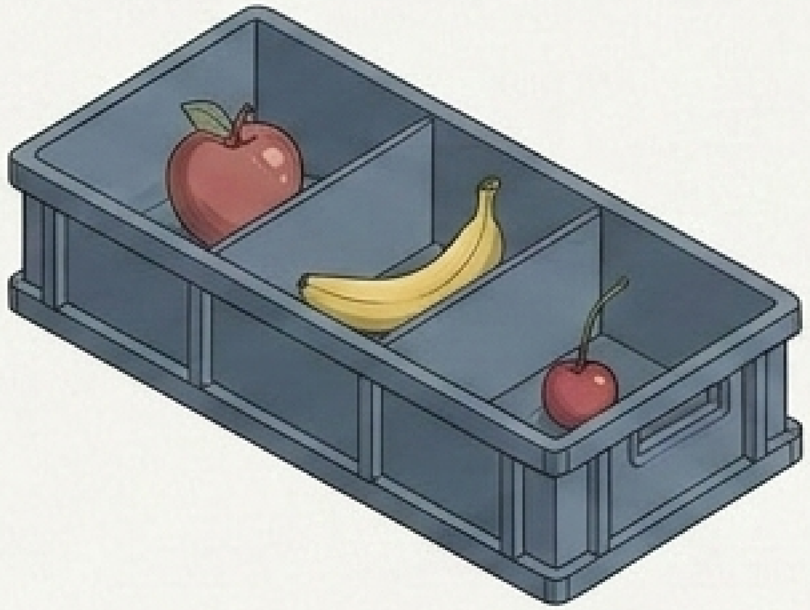


Change

`fruits[0] = "blueberry"`
(Swaps an item)

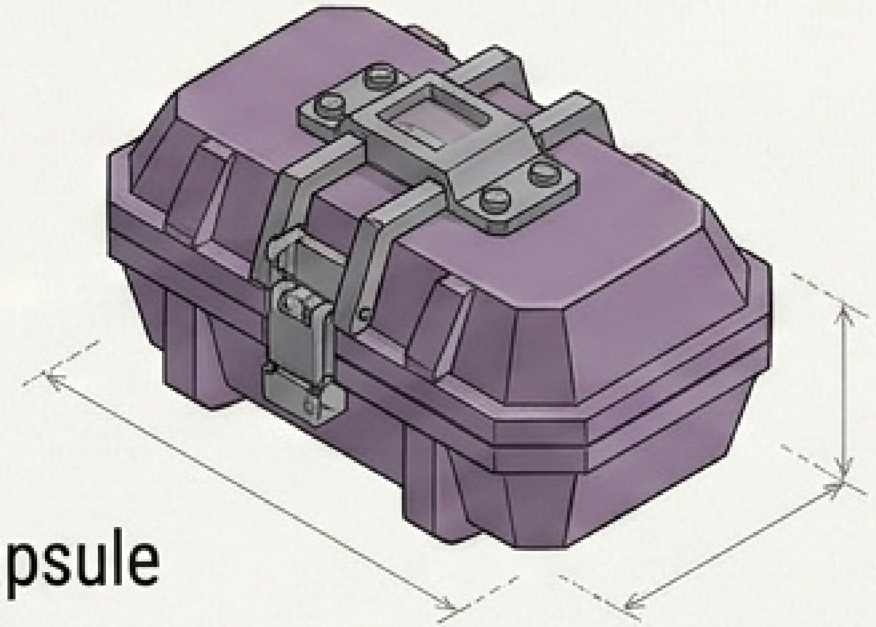
Tuples are sealed, tamper-proof data capsules designed for fixed collections

List [ ]



- Open Crate
- Mutable
- Modified on the fly
- Flexible

Tuple ( )



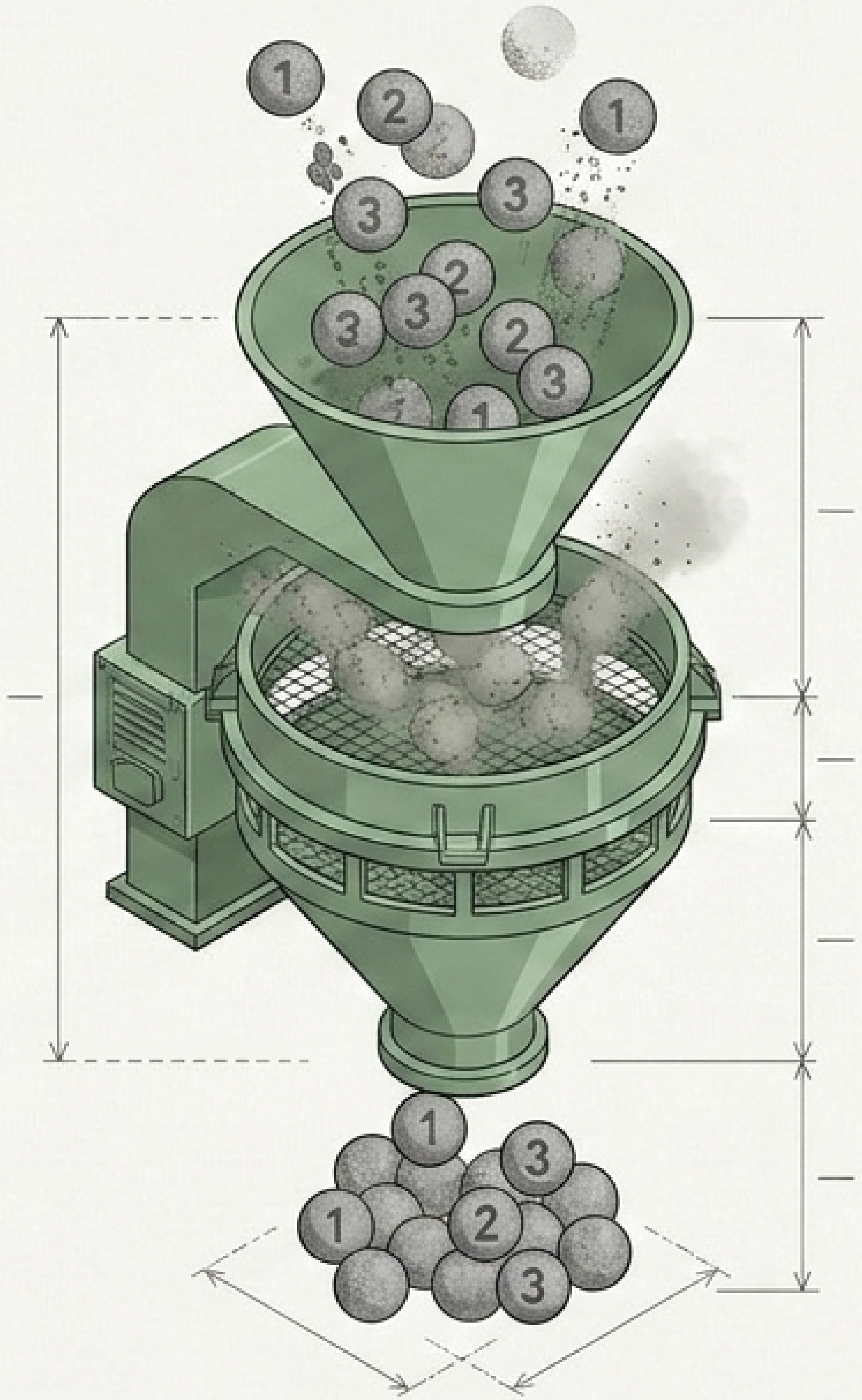
- Sealed Capsule
- Immutable
- Cannot normally be modified
- Safe for fixed collections

```
coordinates = (10, 20)
print(coordinates[1]) # Output: 20
```

Sets act as an automated filtering bouncer that instantly removes duplicate data

Core Mechanics

- Stores unique values.
- Automatically removes duplicates.
- Highly efficient for membership testing.

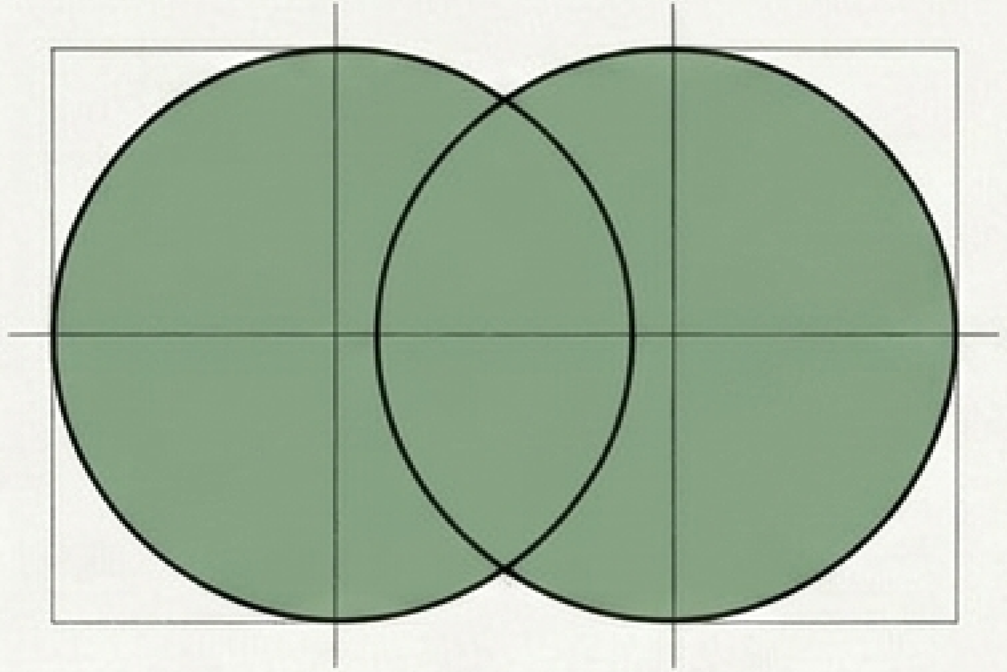


Syntax Window

```
ids = {1, 2, 2, 3, 3, 3}
print(ids) # Result: {1, 2, 3}
```

Comparing data collections requires mathematical set operations

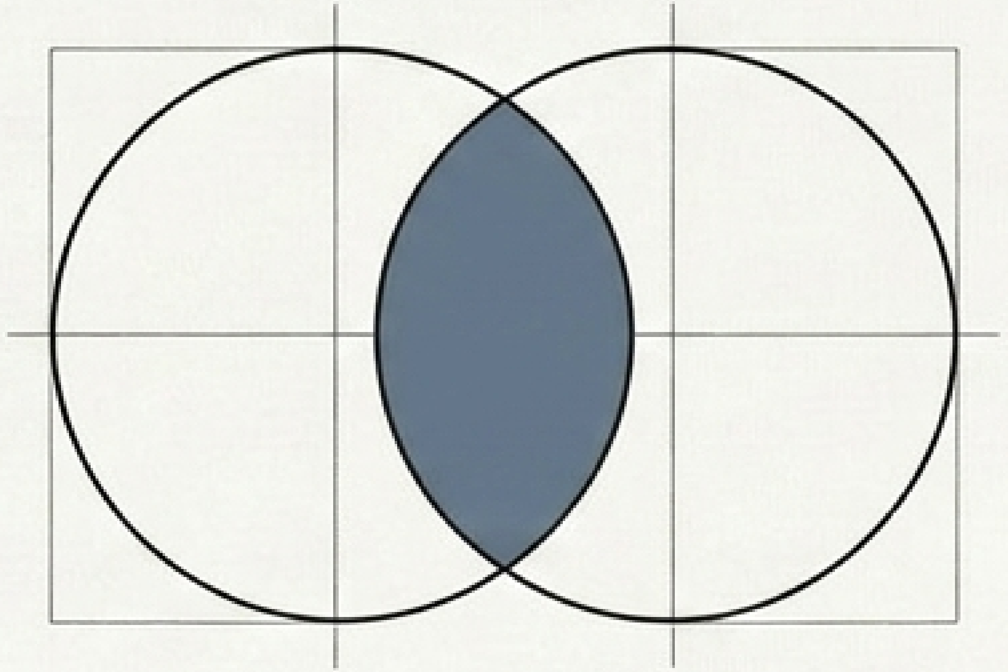
Union (|)



Combines everything unique

```
set1 | set2
```

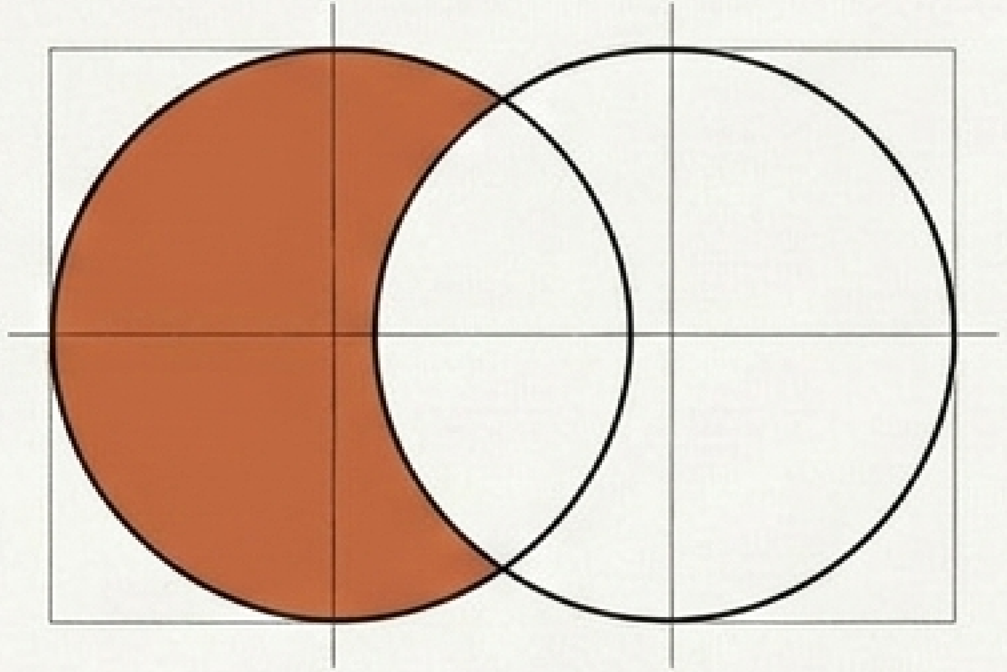
Intersection (&)



Finds common ground

```
set1 & set2
```

Difference (-)



Isolates unique left-side traits

```
set1 - set2
```

These operations are essential when comparing distinct collections of AI training data.

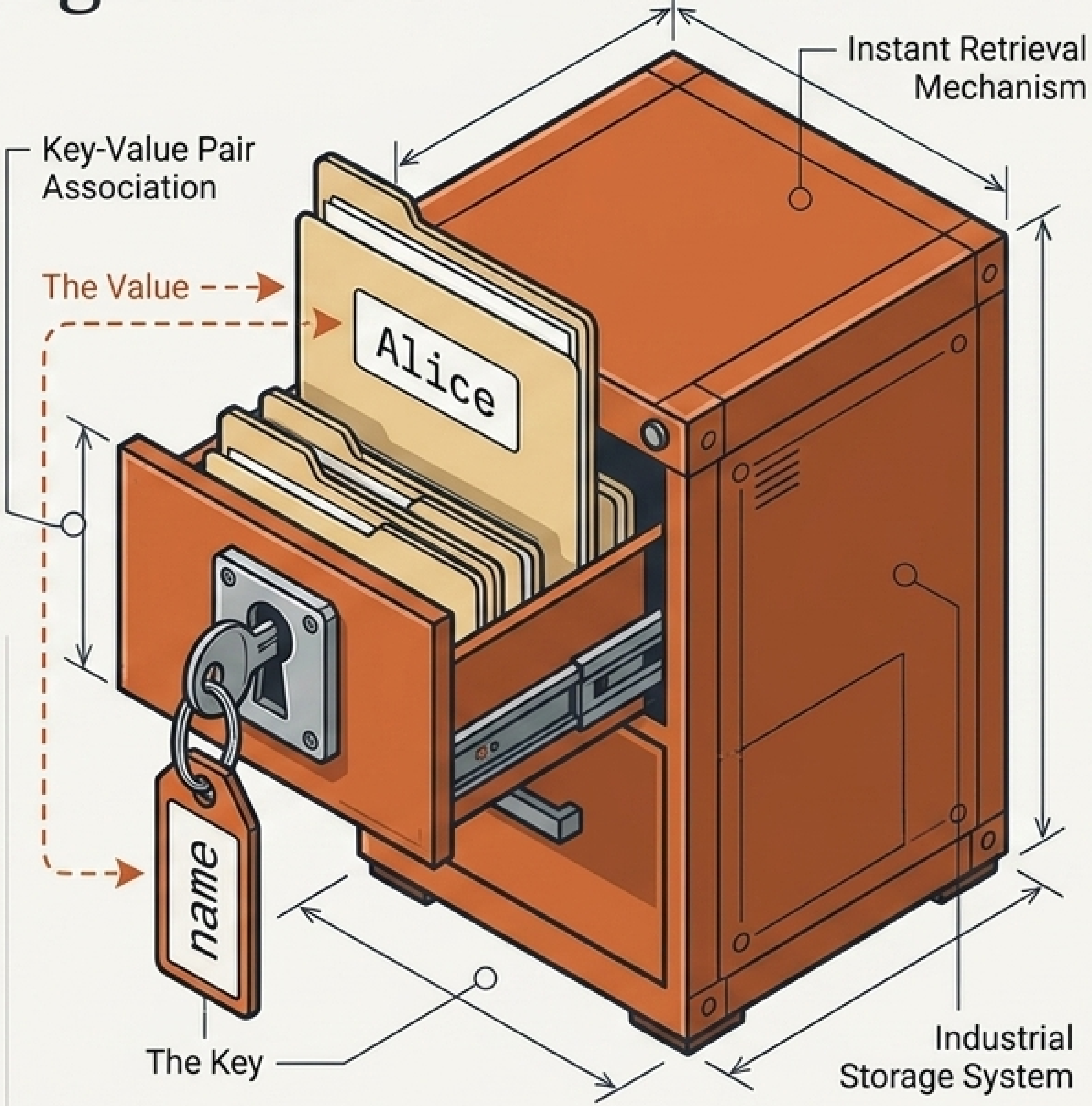
Dictionaries utilize high-tech filing cabinets for instant, label-based retrieval

Core Mechanics

- Stores Key-Value pairs.
- Retrieves data by explicit name rather than numerical order.

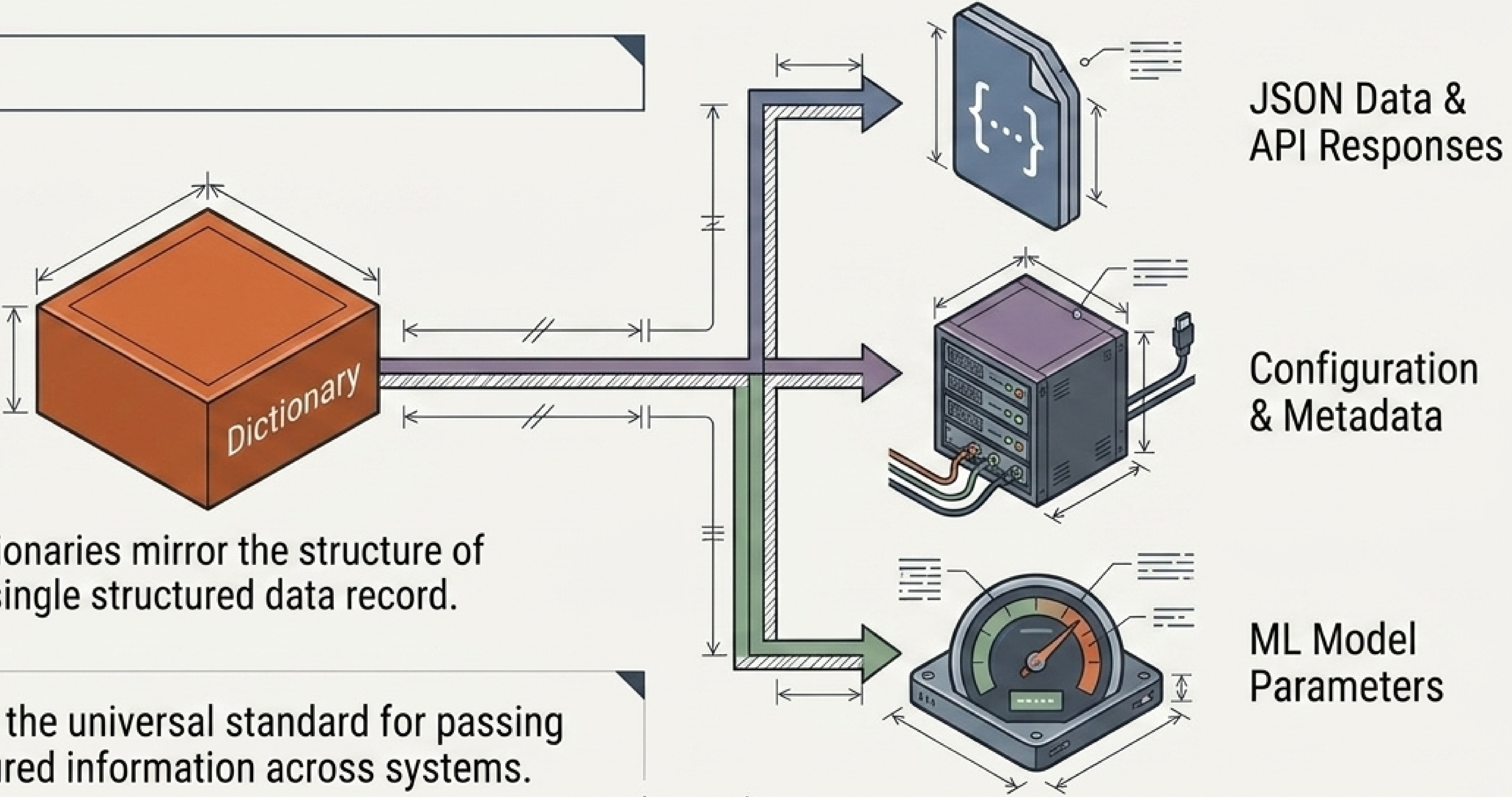
```
person = {"name": "Alice", "age": 25}
print(person["name"]) # Output: Alice
```

Syntax Window



Key-value mapping provides the structural backbone for modern AI configurations

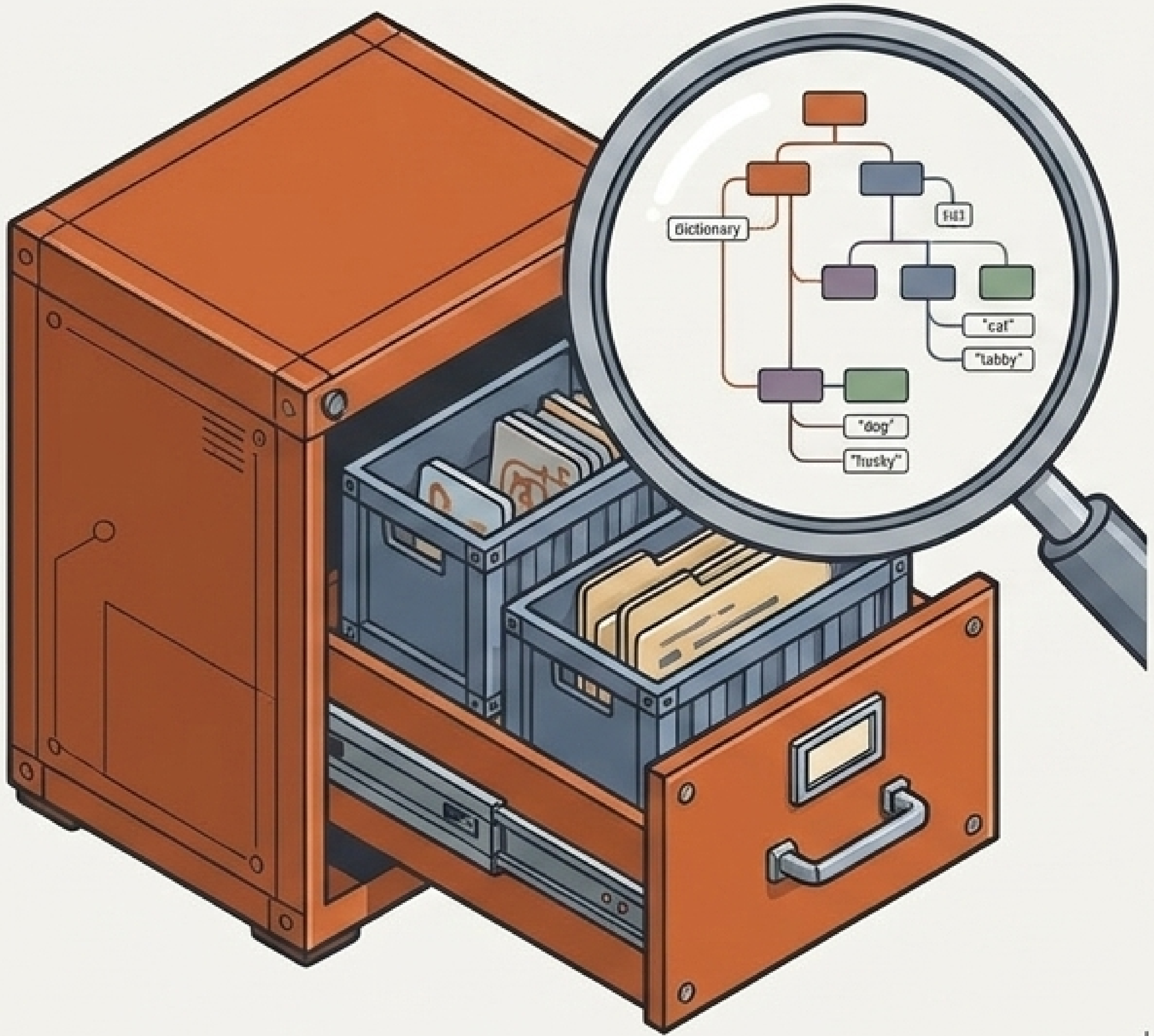
Tree Node



Dictionaries mirror the structure of a single structured data record.

They are the universal standard for passing structured information across systems.

Nesting structures builds the complex, multi-layered data pipelines used in the real world



The Concept

Real-world data isn't flat. You can seamlessly place structures inside other structures.

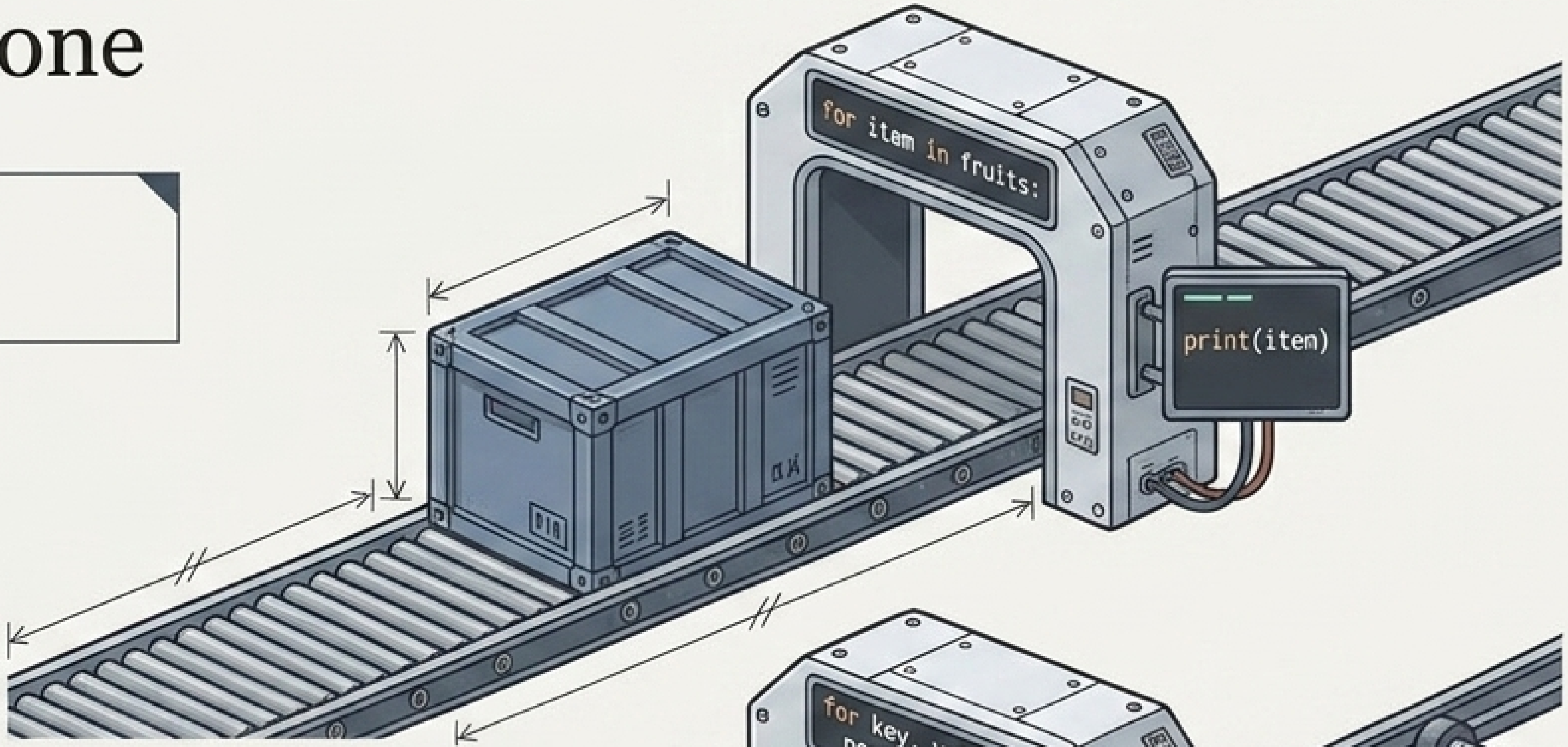
```
dataset = [
   {"id": 1, "labels": ["cat", "tabby"]},
   {"id": 2, "labels": ["dog", "husky"]}
]
```

This nested pattern is the foundational blueprint when fetching dataset records from any web API.

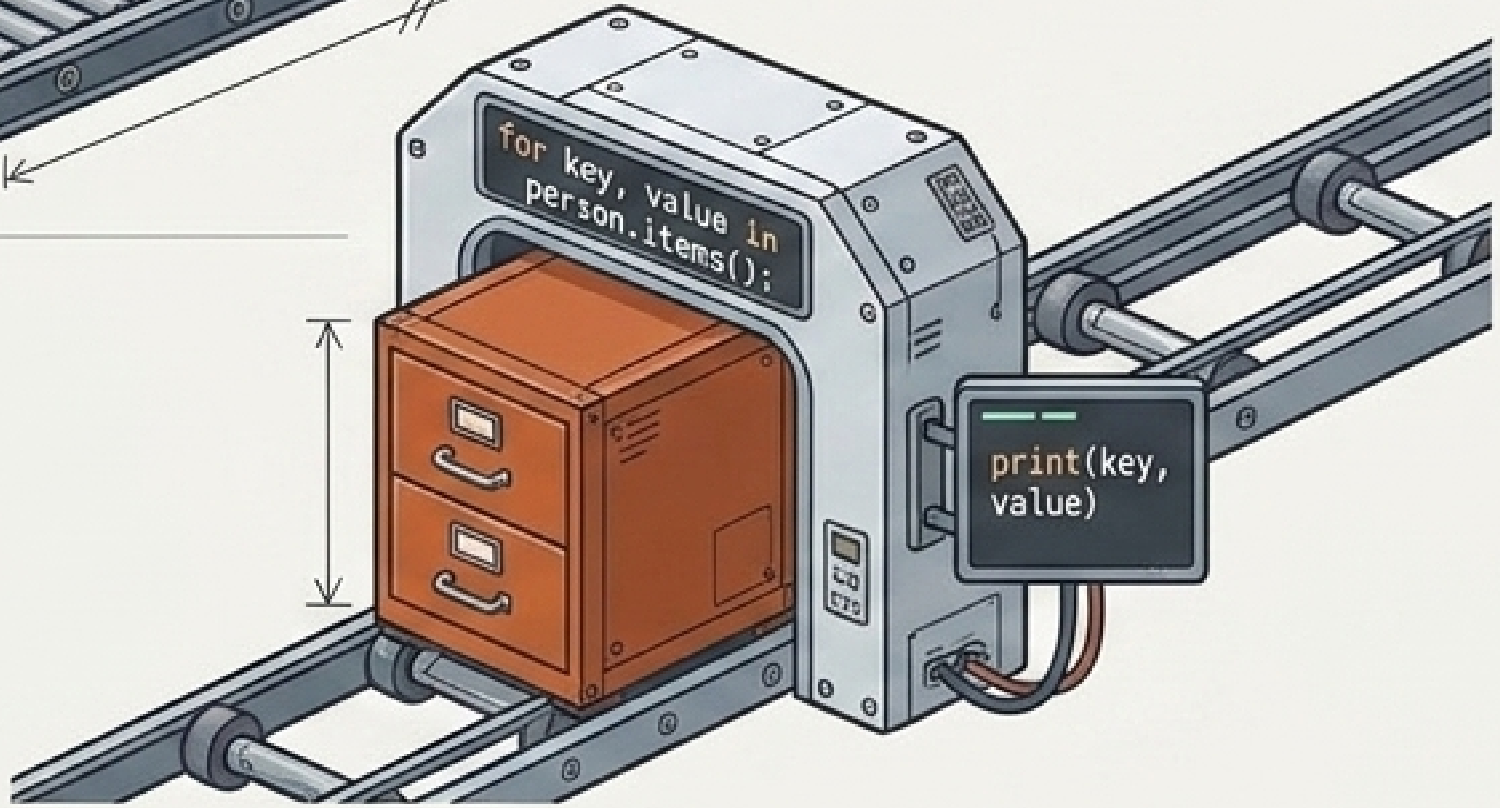
Iteration acts as the processing engine, handling logistics items one by one

Iteration is the fundamental pattern for large-scale data processing in Python.

List Iteration

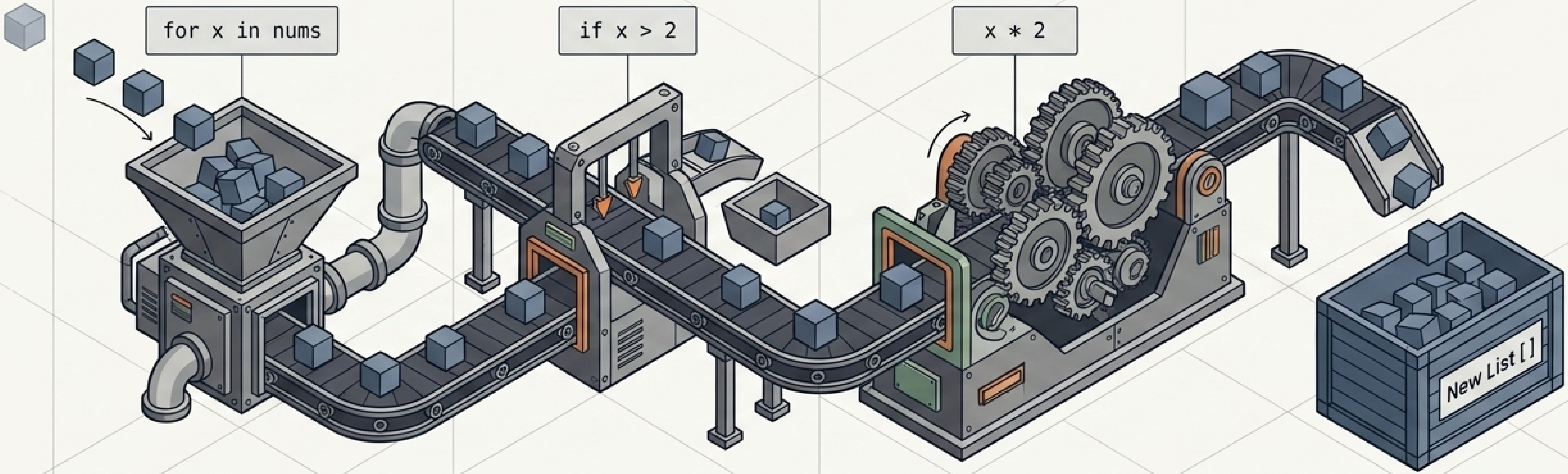


Dict Iteration



List comprehensions function as compact, all-in-one data transformation machines

Python provides a compact syntax to map and filter lists simultaneously.




```
[x * 2 for x in nums if x > 2]
```

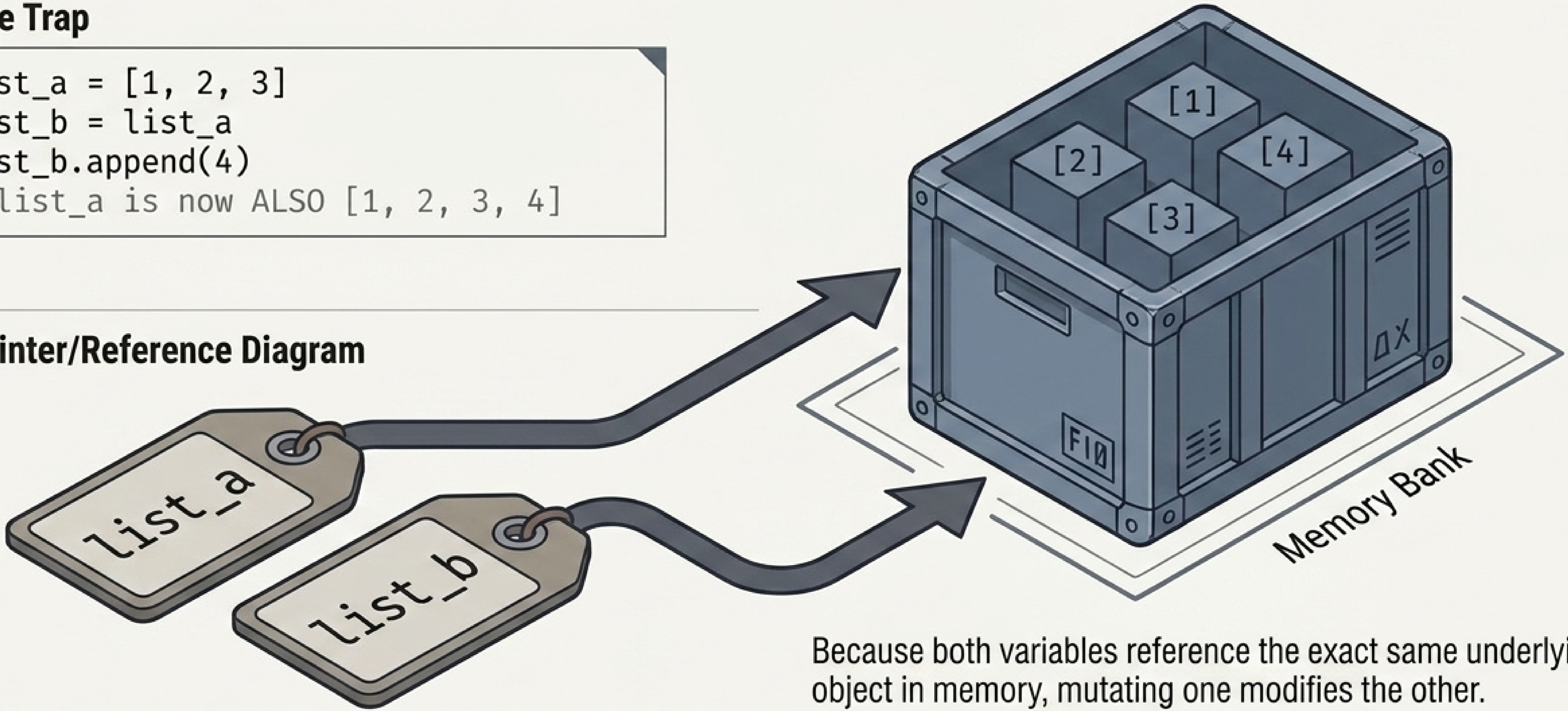
A 4-line loop collapsed into a single line.

The Mutability Trap: Variable names act as directional pointers, not isolated copies

The Trap

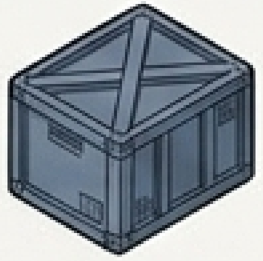
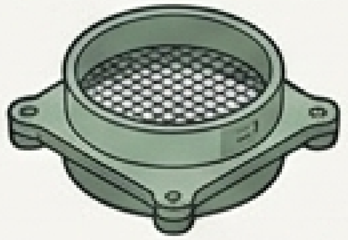
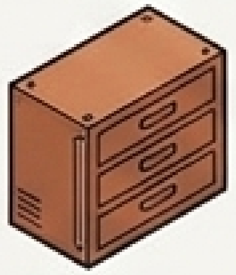
```
list_a = [1, 2, 3]
list_b = list_a
list_b.append(4)
# list_a is now ALSO [1, 2, 3, 4]
```

Pointer/Reference Diagram



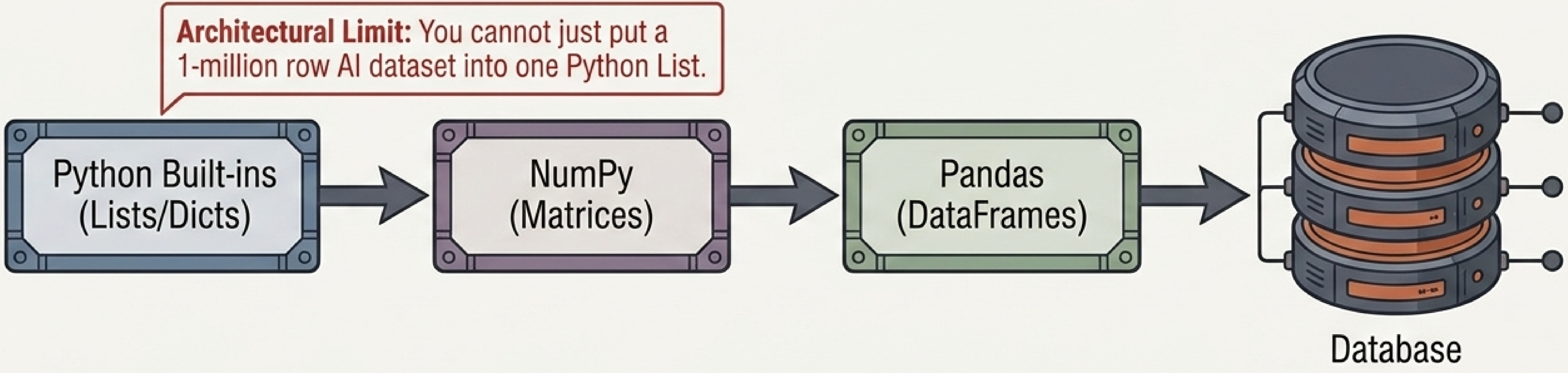
Because both variables reference the exact same underlying object in memory, mutating one modifies the other.

The Master Logistics Decision Matrix

Structure	Ordered	Mutable	Duplicates	Main Use
 `List`	✓	✓	✓	General collections
 `Tuple`	✓	✗	✓	Fixed collections
 `Set`	✗*	✓	✗	Unique values
 `Dictionary`	Key-based	✓	Keys unique	Structured data

* Modern sets have predictable iteration, but should not be treated as strictly ordered data.

The roadmap from built-in Python logic to advanced AI frameworks



Final Takeaway Checklist: Indexing → Slicing → Iteration → Mutability → Nested structures → Comprehensions

\---

Practice Task:

```
students = [{"name": "A", "score": 90}, {"name": "B", "score": 75}]
Write a loop to print only students with a score > 80.
```

(Preparing you for real dataset filtering).